vdot.vv 指令设计说明

本次设计面向香山昆明湖 V2 架构的向量执行后端，新增了固定 8 元素有符号点积指令 vdot.vv。该指令在执行时直接截取两个源向量寄存器 vs1、vs2 的低 8 个 INT8 元素完成乘累加，将精确求和结果写入目标寄存器 vd 的低 32 位，并将高 96 位直接清零。目前，配套的 939 例综合测试程序已在香山处理器与 NEMU 模拟器双端协同仿真环境下全部跑通并完成比对。本报告主要阐述该指令的编码规范、微架构集成方案、数据通路设计以及源码合入清单；具体的环境部署与仿真运行操作已统一归档至项目共用手册。

1 指令规则

1.1 输入范围与完整输出

在当前工程配置下，向量寄存器宽度 VLEN 定为 128 位，单个源寄存器可容纳 16 个字节。vdot.vv 采用固定窗口机制，固定截取低 64 位中的第 0 至第 7 号元素参与点积运算，其中元素 0 对应最低有效字节。每个参与运算的字节均按 8 位二进制补码解析，数值覆盖范围为−128 至 127。

```
acc = 0
for i = 0 .. 7:
    a = signed_int8(vs1[8*i+7 : 8*i])
    c = signed_int8(vs2[8*i+7 : 8*i])
    acc = acc + a * c
vd[31:0] = signed_int32(acc)
vd[127:32] = 0
```

在结果写回策略上，本项目将目标寄存器 vd 的低 32 位划定为结果存放区，用于完整保存 8 项乘积的累加和。虽然输入侧仍按 8 位元素宽度组织，但软件侧在读取运算结果时必须按 32 位有符号整型解析。若累加结果为负数，仅在低 32 位区间内以补码形式呈现，高 96 位依然保持全零。此外，该指令属于纯覆盖型运算，不读取目标寄存器旧值作为累加初值，运算过程不引入舍入或饱和截断，也不会修改 vxrm 与 vxsat 状态寄存器。

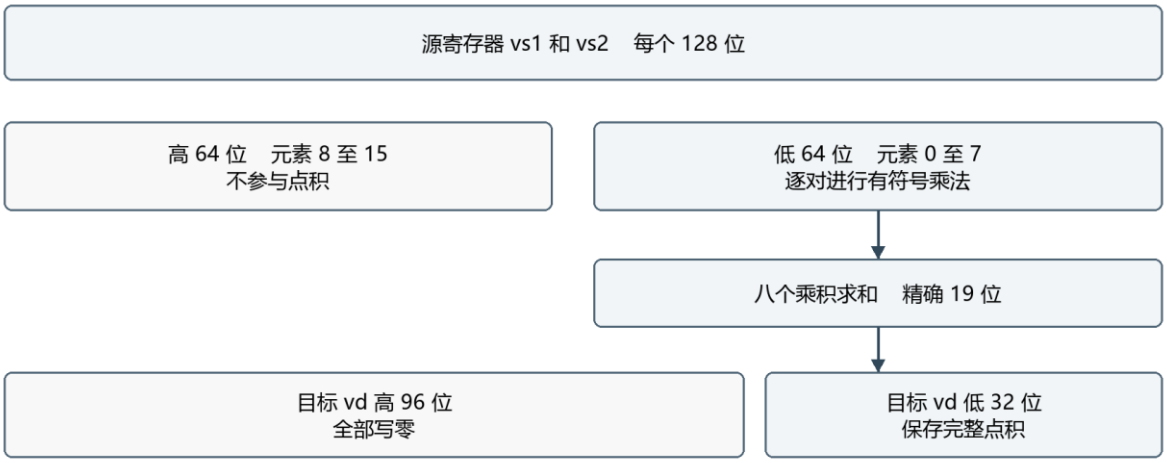


图 1 参与点积的输入范围与目标寄存器布局

1.2 合法向量配置

配置	当前执行要求	硬件执行含义
SEW	e8	输入元素固定为 8 位有符号整数
LMUL	m1	每个源操作数占用单个向量寄存器
vm	1	禁用逐元素掩码控制
vstart	0	不支持从中间元素位置断点恢复
vill	0	当前向量类型配置必须处于合法有效状态
VL	不影响固定运算窗口	已针对 VL 为 0、1、4、8、16 的场景完成验证
vta 和 vma	不影响整寄存器写回	高 96 位强制写零，不保留目标寄存器旧值

执行期间，若处理器检测到 SEW、LMUL、vm、vstart 或 vill 任一配置不满足上表要求，将直接触发非法指令异常。寄存器分配方面，硬件允许目标寄存器与源寄存器重合，也支持将 v0 直接作为目标寄存器，运算结果会整块覆盖目标寄存器。需要说明的是，这种固定 8 元素窗口规则仅针对 vdot.vv 自定义指令生效，常规向量加载与算术指令仍严格受标准 RVV 规则约束。

在开展 VL 小于 8 的边界测试时，测试程序需先在 VL 为 8 或 16 的配置下调用 vle8.v 将参与运算的 8 个字节完整装入源寄存器，随后再改设待测的 VL 值并执行点积；前置的 vle8.v 指令依然遵循标准 RVV 的 VL 长度截断规则。

1.3 机器编码

该指令的汇编助记符格式为 vdot.vv vd, vs2, vs1。当前固定版本采用如下表所示的自定义编码方案。在使用常规汇编器编译时，可直接通过 .word 伪指令内嵌机器码。

位域范围	编码项名称	编码取值与用途说明
31 至 26	funct6	101100
25	vm	1
24 至 20	vs2	第二源向量寄存器编号
19 至 15	vs1	第一源向量寄存器编号
14 至 12	funct3	010
11 至 7	vd	目标向量寄存器编号
6 至 0	opcode	1010111，即 0x57

```
# vdot.vv v4, v9, v8
# vd=v4  vs2=v9  vs1=v8
.word 0xb2942257
```

由于通用反汇编工具尚未录入该自定义指令，反汇编结果中通常会显示为 .word 原始数据或未知指令提示，调试时需对照机器编码表人工核验目标寄存器 vd 与两个源寄存器 vs2、vs1 的编号位置。本报告后续章节统一以 vdot.vv 指代该软件接口。

2 微架构与控制

2.1 在香山中的执行位置

前端译码模块识别出点积指令编码后，由寄存器重命名模块为目标分配新的物理寄存器，并交由向量调度队列等待两个源操作数就绪。指令发射后经由 VFEX0 通道进入点积功能单元 VDotU，通过内部两级流水数据通路完成乘加运算，最终产出完整的 128 位结果并送入现有的向量写回通路，由 ROB 按程序顺序完成架构提交。

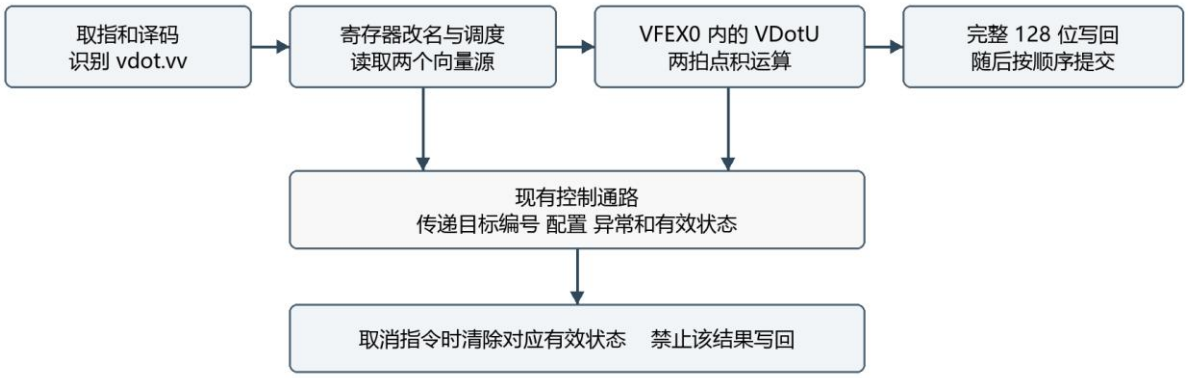


图 2 点积功能单元与香山执行控制结构的连接

在模块封装上，VDotU 直接继承自 VecPipedFuncUnit 基类，全面复用香山原有的源寄存器读取、目标物理寄存器编号传递、有效状态标志、异常透传及写回控制逻辑。点积单元属于纯算术部件，不直接访问存储器，也不绕过流水线修改架构寄存器。由于 VFEX0 通道同时挂载了原有的常规向量运算部件，点积指令与同通道向量算术指令共享该端口的发射资源。

2.2 功能单元配置

项目	实际配置	作用
内部延迟	CertainLatency(2)	驱动控制流水线与两级数据计算路径严格对齐
数据宽度	destDataBits=128	单次写回直接完整覆盖整个 128 位向量寄存器
源数据	vs1=inData.src(0)；vs2=inData.src(1)	从发射通道读取两个 128 位源向量操作数
旧目标	SrcType.no	译码阶段不建立对目标寄存器旧值的数据依赖
完整写入	isDependOldVd=false；isWritePartVd=false	显式关闭旧目标读取依赖与部分写入依赖
写回许可	writeVecRf=true；writeV0Rf=true	复用现有的通用向量寄存器与 v0 写回通路
执行位置	VFEX0	在既有的 VFEX0 执行通道内实例化 VdotCfg

尽管公共接口层仍保留了旧目标寄存器的读取槽位，但译码逻辑在解析点积指令时已将该源操作数显式标记为不使用。当遇到目标寄存器与源寄存器同名的重叠场景时，寄存器重命名机制会自动区分待读取的旧源物理寄存器与待写入的新目标物理寄存器；功能单元先采样输入源数据，经过流水计算后再生成全新的 128 位完整目标结果。

2.3 有效状态与异常对齐

在独立算术核 VDotUnit 内部，电路利用 fire 信号锁存第一级乘积，并将有效状态顺延两级流水。接入香山后端封装层后，内部 dot.io.fire 直接由输入端的 io.in.valid 驱动，而对外的 io.out.valid 则统一交由继承自父类的 HasPipelineReg 流水寄存器链管理，从而避免底层算术核的独立 valid 信号与处理器主控流水线状态发生脱节。

在写回数据端，封装层输出直接硬连线至 dot.io.vd，旁路了常规逐元素向量运算所需的掩码拼合与尾部保留逻辑。为了保证异常精确对齐到指令，包装层在输出级利用随流水线同步传递的 outVecCtrl、outVm 和 outVstart 信号做合法性判别，确保触发的异常严格归属于当前已完成计算的指令。一旦发生后端冲刷取消或写回受阻，运算结果是否有效完全服从香山现有的流水线控制逻辑，工程上无需将底层独立算术模块额外设计为带反压握手的停顿队列。

3 数据通路

3.1 两级计算结构



图 3 当前 Chisel 与 Verilog 对应的数据通路

第一级流水负责字节拆分与有符号乘法。电路从源操作数中拆出低 8 对字节并执行并行有符号乘法；8 位乘 8 位的精确乘积可由 16 位有符号数完整表示，随后在锁存入第一级乘积寄存器前统一将符号位扩展至 19 位。针对负乘积必须严格执行符号位扩展，不可用零扩展替代。

第二级流水负责对 8 路乘积归约求和。当前正式合入香山的配置固定为 tree=0，即平衡进位传播加法树结构。为保持参数化源码接口一致，底层保留了 16 个加法树输入端口，但将索引 8 至 15 号输入在乘法前端直接置零，因此实际参与归约的依然只有低 8 个源元素。加法树各级统一采用 19 位位宽运算，求和结果锁存入第二级寄存器，出拍后再符号扩展至 32 位，并在高 96 位拼接全零以构成 128 位输出。

在具体实现上，Chisel 版本的平衡加法树通过逐层将相邻两项相加完成归约；参数化源码中虽保留了另一套压缩树结构实现，但未作为本次合入香山的正式配置。由于加法树全程在 19 位位宽下运行，所有合法输入产生的乘积及其中间累加和均在可精确表示范围内，不会发生最终结果溢出。

3.2 位宽推导

数值	最小值	最大值
单个 INT8 输入	−128	127
单个精确乘积	−16256	16384
八项乘积之和	−130048	131072
18 位有符号数	−131072	131071
19 位有符号数	−262144	262143

从极值边界来看，8 路乘积之和的正向最大值为 8×(−128)×(−128)=131072，恰好比 18 位有符号数的正数上限 131071 大 1；负向最小值为 8×(−128)×127=−130048。由此可见，累加通路至少需要 19 位有符号位宽方可无损覆盖全部结果；最终写入目标低 32 位时仅需做符号位扩展，不涉及任何截断、舍入或饱和处理。

3.3 两级时序的含义

时序节点	数据通路动作	控制有效状态
输入采样前	两个源向量数据与指令控制信号同步就绪	输入有效
第一个时钟沿	8 路精确乘积完成符号扩展并锁存入第一级寄存器	第一级有效
第二个时钟沿	19 位乘积归约总和锁存入第二级输出寄存器	结果可用
写回与提交	通过现有处理器写回结构更新目标寄存器并按序提交	受写回与取消控制

上述两级时序仅界定功能单元内部从输入采样到计算结果就绪的硬件流水结构。而在实际程序运行过程中，端到端耗时还包含了向量配置、数据装载、发射调度、标量读回以及循环分支等外围指令开销；关于完整基准程序的实测周期与加速比结论，请参见专项报告《3.3_延迟加速分析报告》，不能仅凭单元内部的 2 级流水直接推算。

4 实现文件与接入修改

本项目交付的全部配套源码、补丁与测试文件均归档于相关证明材料文件夹中。表 6 所列相对路径均以该目录为基准起算。

文件位置	工程角色与当前用途
rtl/xs_vdot_unit.sv	独立封装的 SystemVerilog 点积功能单元
rtl/chisel/vdot/vdot/src/vdot/VDotUnit.scala	与 Verilog 版本严格对应的 Chisel 点积运算实现
rtl/xiangshan-patch/VDotU.scala	面向香山后端的数据通路与异常判别包装层
rtl/xiangshan-patch/install.sh	自动化补丁脚本，将正式接入修改合入固定版香山源码

文件位置	工程角色与当前用途
rtl/xs_wrapper_generated/VDotU.sv	由接入包装层生成的 Verilog 文件，用于独立功能检查
gen/chisel/VDotUnit_t0_n8_o32_a19.sv	按当前固化参数生成的 Chisel 算术电路网表
nemu/apply_nemu_patch.py	NEMU 补丁脚本，注入自定义指令译码、执行及比对状态传递逻辑
examples/minimal_vdot.S	演示软件侧指令调用与 128 位完整结果比对的裸机汇编示例

在执行完整处理器构建时，自动化脚本 install.sh 会在工作区的 XiangShan 源码副本中对下表所列文件实施打补丁修改。表中 backend 相关路径均从 src/main/scala/xiangshan 起算；构建工作区内的源码副本则由 sources 目录中存放的固定版本本解压生成。

香山源码位置	具体接入与修改内容
backend/decode/Instructions.scala	新增 vdot.vv 机器编码匹配规则
backend/decode/VecDecoder.scala	配置两个向量源、目标寄存器及点积功能单元类型映射
backend/decode/DecodeUnit.scala	解除对旧目标寄存器读取与部分写入的流水线依赖
backend/decode/VecExceptionGen.scala	新增固定窗口模式及非法向量配置的异常检查逻辑
backend/fu/FuType.scala	注册点积功能单元类型并归入向量指令分类
backend/fu/FuConfig.scala	定义两级固定延迟配置、向量控制信号与写回总线连接
Parameters.scala	将 VdotCfg 配置挂载至 VFEX0 执行通道
backend/fu/wrapper/VDotU.scala	实例化底层点积运算单元并连接输出结果与异常信号
src/main/scala/vdot/VDotUnit.scala	合入实际参与香山完整编译的 Chisel 点积运算源码

5 软件接口与验证对应

5.1 软件读取方式

指令调用示例见 examples/minimal_vdot.S。软件在 e8/m1 装入至少 8 个字节后执行点积；读取标量结果时先切换 e32，再用 vmv.x.s 读取低 32 位，下一次点积前恢复 e8。完整 128 位检查采用 vs1r.v 保存后分两次 64 位比较。当前矩阵接口定义位于 applications/gemm/include/vdot_gemm.h，通用接口源码、编译输入及 36 次处理器计时见 applications/gemm/measured_20261026。

5.2 已完成结果与对应材料

验证内容	材料或结果	当前结果
正数、负数与边界值	2.3.1、2.3.2、2.3.3	46、48、45 组随综合程序通过
独立电路运算	results/basic_test.txt	两种算术电路均通过 939 组输入
完整处理器功能	results/cluster/20261019	综合程序及配置、异常专项检查通过

验证内容	材料或结果	当前结果
包装层检查	results/recorded_nemu/fixed_wrapper.json	96 项配置、时序、结果及错误识别检查通过
NEMU 扩展及协同验证	3.1 报告	完整 128 位目标与架构状态同步比对
当前处理器计时	applications/gemm/与 experiments/throughput/内的 measured_20261026；见 3.3 报告	36 次矩阵和 216 次独立点积，共 252 次

基础验证完成 939 组完整 128 位比较、96 项包装层检查及处理器功能和异常专项检查。通用矩阵接口 36 次、独立点积 216 次处理器计时已全部通过。指令语义与完整处理器方法见 3.1 报告，当前性能见 3.3 报告。基础结果使用 scripts/check_recorded_results.py，当前计时使用 scripts/check_measurements.py 复算，命令见共用手册第 3、6、7、9 章。

6 适用范围

本设计交付的正式硬件接口严格限定为 VLEN=128、低 8 个有符号 INT8 输入、e8/m1 模式、非掩码 vm=1 且 vstart=0 条件下的 vdot.vv 指令。硬件不支持将结果累加至旧目标寄存器，也不支持可变元素位宽、运行时动态指定元素数量或中途断点恢复。完整处理器的回归结果证明随附测试激励与基准程序已全部通过验证，但受限于有限测试规模，流水线停顿、异常冲刷取消及乱序执行交织的所有极端时序组合仍无法通过定向测试穷尽。

7 参考文献

香山公开源码仓库：https://github.com/OpenXiangShan/XiangShan

NEMU 公开源码仓库：https://github.com/OpenXiangShan/NEMU

RISC-V 向量扩展规范：https://docs.riscv.org/reference/isa/unpriv/v-st-ext